Always See Your Specs Fail



Tests, like implementation code, can contain bugs, but we don't have tests for our tests! So, check each test by making it go red, confirming it fails the way you expect, and making it pass.

05-unit-specs/04/expense_tracker/app/api.rb

```
post '/expenses' do
  status 404

  expense = JSON.parse(request.body.read)
  result = @ledger.record(expense)
  JSON.generate('expense_id' => result.expense_id)
end
```

Now, rerunning the spec fails as expected:

```
$ bundle exec rspec spec/unit/app/api_spec.rb
« truncated »

Failures:

  1) ExpenseTracker::API POST /expenses when the expense is successfully
  recorded responds with a 200 (OK)
     Failure/Error: expect(last_response.status).to eq(200)

       expected: 200
            got: 404

       (compared using ==)
     # ./spec/unit/app/api_spec.rb:41:in `block (4 levels) in
     <module:ExpenseTracker>'

Finished in 0.03479 seconds (files took 0.14115 seconds to load)
4 examples, 1 failure, 2 pending

Failed examples:

rspec ./spec/unit/app/api_spec.rb:33 # ExpenseTracker::API POST /expenses
when the expense is successfully recorded responds with a 200 (OK)

Randomized with seed 32399
```

Make sure you un-break the app before continuing. Once you do that, you'll be back to two passing specs. Now, we can turn our attention to test maintainability. There's a lot of duplicated code in these two test cases. Before moving on to the next section, consider how you might make the code a little less repetitive.